



COLEGIO APEC
Fernando Arturo de Meriño

Colegio Apec Fernando Arturo de Meriño

**Enmanuel Molina Abreu y Dohrian Miguel Peña
Castro**

20180786302@cafam.edu.do y
20190804901@cafam.edu.do

**Desarrollo de aplicaciones y administración de
empresas**

SOLID

5to de informática

Víctor Recio

Objetivo de la tarea

Al finalizar esta actividad, el estudiante será capaz de comprender los principios SOLID, identificar problemas comunes en código orientado a objetos y aplicar técnicas de refactorización para mejorar la mantenibilidad, escalabilidad y claridad del software.

Investigación y análisis

Investiga y responde:

1. ¿Qué significa SOLID?

Explica qué representa cada letra:

SOLID es un conjunto de cinco principios de diseño orientado a objetos propuestos por Robert C. Martin. Su objetivo es crear software más mantenible, flexible, escalable y fácil de entender.

S → Single Responsibility Principle (Principio de Responsabilidad Única)

- **Definición:** Una clase debe tener una sola responsabilidad o una única razón para cambiar.
- **Idea principal:** Cada clase debe encargarse de una sola tarea específica.
- **Ejemplo:** Si una clase Factura calcula totales, genera reportes y guarda datos en la base de datos, tiene demasiadas responsabilidades. Lo correcto sería separar esas funciones en clases distintas.
- **Beneficio:** Facilita el mantenimiento y reduce el impacto de los cambios.

O → Open/Closed Principle (Principio Abierto/Cerrado)

- **Definición:** Las entidades de software deben estar abiertas para su extensión, pero cerradas para su modificación.
- **Idea principal:** Se debe poder agregar nuevo comportamiento sin modificar el código existente.
- **Ejemplo:** Si una aplicación calcula descuentos para distintos tipos de clientes, en lugar de modificar constantemente una clase existente, se pueden crear nuevas clases que implementen diferentes estrategias de descuento.
- **Beneficio:** Reduce errores al agregar nuevas funcionalidades.

L → Liskov Substitution Principle (Principio de Sustitución de Liskov)

- **Definición:** Los objetos de una clase derivada deben poder sustituir a los objetos de su clase base sin alterar el funcionamiento correcto del programa.
- **Idea principal:** Una subclase debe comportarse de manera compatible con la clase padre.

- **Ejemplo:** Si una clase Ave tiene el método volar(), una subclase Pingüino no debería heredar ese comportamiento porque los pingüinos no vuelan. En este caso, el diseño de la jerarquía debería replantearse.
- **Beneficio:** Garantiza que la herencia se use correctamente y evita comportamientos inesperados.

I → Interface Segregation Principle (Principio de Segregación de Interfaces)

- **Definición:** Ningún cliente debe depender de métodos que no utiliza.
- **Idea principal:** Es mejor tener varias interfaces pequeñas y específicas que una interfaz grande con muchas funciones.
- **Ejemplo:** En lugar de una interfaz Trabajador con métodos trabajar(), comer() y volar(), se pueden crear interfaces separadas para cada comportamiento y que cada clase implemente solo las que necesita.
- **Beneficio:** Reduce el acoplamiento y mejora la claridad del código.

D → Dependency Inversion Principle (Principio de Inversión de Dependencias)

Definición: Los módulos de alto nivel no deben depender de módulos de bajo nivel; ambos deben depender de abstracciones.

Idea principal: Las dependencias deben dirigirse hacia interfaces o abstracciones, no hacia implementaciones concretas.

Ejemplo: Una clase Notificador no debería depender directamente de una clase EmailService. En su lugar, debería depender de una interfaz INotificacion, permitiendo usar correo electrónico, SMS o cualquier otro canal sin modificar la clase principal.

Beneficio: Aumenta la flexibilidad, facilita las pruebas y reduce el acoplamiento entre componentes.

Resumen rápido

Letra	Principio	Idea clave
S	Single Responsibility	Una clase, una responsabilidad.
O	Open/Closed	Extender sin modificar.

L	Liskov Substitution	Las subclasses deben poder reemplazar a sus clases base.
I	Interface Segregation	Interfaces pequeñas y específicas.
D	Dependency Inversion	Depender de abstracciones, no de implementaciones.

Estos principios ayudan a desarrollar sistemas más robustos, reutilizables y fáciles de mantener a largo plazo.

2. ¿Por qué SOLID es importante en la Programación Orientada a Objetos?

Los principios **SOLID** son importantes porque ayudan a diseñar software que sea más fácil de mantener, ampliar y comprender. A medida que un proyecto crece, también aumenta su complejidad. SOLID proporciona reglas que permiten organizar mejor el código y evitar problemas comunes en el desarrollo de software.

¿Cómo ayuda SOLID a mantener proyectos grandes?

En proyectos grandes suelen participar muchos desarrolladores y existir miles de líneas de código. SOLID ayuda a:

- Dividir el sistema en componentes con responsabilidades claras.
- Reducir la dependencia entre módulos.
- Facilitar la incorporación de nuevas funcionalidades.
- Evitar que un cambio en una parte del sistema afecte a otras áreas.
- Mejorar la organización y comprensión del código.

Gracias a estos principios, los proyectos pueden evolucionar durante años sin volverse inmanejables.

¿Qué problemas aparecen cuando no se usan estos principios?

Cuando SOLID no se aplica, suelen aparecer problemas como:

- **Código difícil de entender:** las clases realizan demasiadas tareas.
- **Alto acoplamiento:** los componentes dependen demasiado unos de otros.
- **Duplicación de código:** se repiten funcionalidades similares en varias partes.
- **Mayor cantidad de errores:** un pequeño cambio puede afectar muchas áreas del sistema.

- **Dificultad para agregar nuevas funciones:** se requiere modificar código existente constantemente.
- **Pruebas complicadas:** resulta difícil aislar componentes para verificar su funcionamiento.

Estos problemas aumentan el costo y el tiempo de mantenimiento del software.

Influencia de SOLID en diferentes aspectos 1. Mantenimiento

SOLID hace que el código sea más fácil de corregir y actualizar.

Beneficios:

- Los cambios se realizan en módulos específicos.
- Se reducen los efectos secundarios.
- Los errores son más fáciles de localizar.

Ejemplo: Si una clase tiene una única responsabilidad, modificar una función no afectará otras tareas relacionadas.

2. Reutilización

SOLID fomenta la creación de componentes independientes que pueden utilizarse en distintos proyectos.

Beneficios:

- Menos duplicación de código.
- Mayor aprovechamiento de clases e interfaces.
- Desarrollo más rápido.

Ejemplo: Una interfaz de notificaciones puede reutilizarse para correo electrónico, SMS o mensajes push.

3. Escalabilidad

La escalabilidad consiste en poder ampliar el sistema sin reescribir grandes partes del código.

Beneficios:

- Nuevas funcionalidades pueden añadirse fácilmente.
- El sistema crece de forma ordenada.
- Se reduce el riesgo de introducir errores al expandir el proyecto.

Ejemplo: Aplicando el principio Abierto/Cerrado (O), se pueden agregar nuevos tipos de usuarios sin modificar las clases existentes.

4. Pruebas

SOLID facilita la creación de pruebas unitarias y automatizadas.

Beneficios:

- Los componentes pueden probarse de forma independiente.
- Es más sencillo simular dependencias mediante interfaces.
- Se detectan errores más rápidamente.

Ejemplo: Una clase que depende de una abstracción puede probarse usando objetos simulados (*mocks*) en lugar de servicios reales.

5. Trabajo en equipo

Cuando varios desarrolladores trabajan en un mismo proyecto, SOLID mejora la colaboración.

Beneficios:

- Cada miembro puede trabajar en módulos diferentes sin generar conflictos.
- El código resulta más comprensible para nuevos integrantes.
- Se establecen estándares de diseño más claros.

Ejemplo: Un desarrollador puede implementar una nueva funcionalidad mientras otro corrige errores en otro módulo, sin interferir entre sí.

Conceptos clave

1. Acoplamiento

El **acoplamiento** es el grado de dependencia que existe entre diferentes clases o módulos de un sistema.

- **Bajo acoplamiento:** los componentes son más independientes y fáciles de modificar.
- **Alto acoplamiento:** un cambio en una parte del sistema puede afectar muchas otras partes.

Ejemplo: Si una clase depende directamente de varias clases específicas, tiene un alto acoplamiento.

2. Cohesión

La **cohesión** indica qué tan relacionadas están las responsabilidades de una clase o módulo.

- **Alta cohesión:** la clase realiza tareas relacionadas entre sí.
- **Baja cohesión:** la clase realiza muchas tareas diferentes y poco relacionadas.

Ejemplo: Una clase llamada Calculadora que solo realiza operaciones matemáticas tiene alta cohesión.

3. Refactorización

La **refactorización** es el proceso de mejorar la estructura interna del código sin cambiar su comportamiento externo.

Su objetivo es:

- Hacer el código más limpio.
- Facilitar el mantenimiento.
- Reducir la complejidad.

Ejemplo: Dividir una función muy grande en varias funciones más pequeñas y claras.

4. Escalabilidad

La **escalabilidad** es la capacidad de un sistema para crecer y adaptarse a nuevas necesidades sin perder rendimiento o estabilidad.

Ejemplo: Una tienda en línea que puede agregar nuevos productos, usuarios y funcionalidades sin necesidad de reconstruir todo el sistema.

5. Mantenibilidad

La **mantenibilidad** es la facilidad con la que un software puede corregirse, actualizarse o mejorarse después de haber sido desarrollado.

Un sistema mantenible:

- Es fácil de entender.
- Tiene código organizado.
- Permite realizar cambios con poco riesgo.

Ejemplo: Corregir un error rápidamente porque el código está bien documentado y estructurado.

6. Abstracción

La **abstracción** consiste en mostrar solo la información necesaria y ocultar los detalles internos de funcionamiento.

Permite centrarse en **qué hace** un objeto y no en **cómo lo hace**.

Ejemplo: Cuando utilizamos un botón en una aplicación, sabemos que realiza una acción al presionarlo, aunque no conozcamos todo el código interno que lo hace funcionar.

7. Dependencias

Las **dependencias** son las relaciones en las que una clase o módulo necesita otro componente para funcionar.

Ejemplo: Una clase Pedido que utiliza una clase BaseDeDatos para guardar información depende de ella.

En buenas prácticas de diseño, se busca que las dependencias se basen en **abstracciones** (interfaces) y no en implementaciones concretas, para que el sistema sea más flexible y fácil de modificar.

Resumen

Concepto	Definición breve
Acoplamiento	Nivel de dependencia entre módulos o clases.
Cohesión	Grado en que las responsabilidades de una clase están relacionadas.
Refactorización	Mejora del código sin cambiar su funcionamiento.
Escalabilidad	Capacidad de crecer y adaptarse a nuevas necesidades.
Mantenibilidad	Facilidad para corregir, actualizar y mejorar el software.
Abstracción	Ocultar detalles internos y mostrar solo lo esencial.
Dependencias	Relaciones donde un componente necesita otro para funcionar.

📖 Parte 1 — Principio S: Single Responsibility Principle (SRP)

✗ Código "malo"

```
public class Reporte
{
    public void GenerarReporte()
    {
        Console.WriteLine("Generando reporte...");
    }

    public void GuardarEnArchivo()
    {
        Console.WriteLine("Guardando archivo...");
    }

    public void EnviarPorCorreo()
    {
        Console.WriteLine("Enviando correo...");
    }
}
```

🔍 Problema

La clase `Reporte` tiene tres responsabilidades distintas: generar, guardar y enviar. Cualquier cambio en una de ellas puede afectar a las demás.

☑ Código refactorizado

```
public class GeneradorReporte
{
    public void Generar()
    {
        Console.WriteLine("Generando reporte...");
    }
}

public class GuardadorArchivo
{
    public void Guardar()
    {
        Console.WriteLine("Guardando archivo...");
    }
}

public class EnviadorCorreo
{
    public void Enviar()
    {
        Console.WriteLine("Enviando correo...");
    }
}
```

Preguntas de análisis

1. ¿Qué problema tenía el diseño original?

La clase concentraba múltiples responsabilidades en un solo lugar, dificultando el mantenimiento. Un cambio en el envío de correos, por ejemplo, obligaba a tocar la misma clase que generaba el reporte.

2. ¿Qué ventajas aporta la refactorización?

Cada clase tiene una única razón para cambiar. Mejora la organización, la reutilización y la capacidad de probar cada componente de forma independiente.

3. ¿Qué ocurriría si el sistema crece?

La clase original se volvería cada vez más compleja y difícil de mantener, acumulando lógica de negocios no relacionada entre sí.

Parte 2 — Principio O: Principio Abierto/Cerrado (OCP)

Código "malo"

```
public class Descuento
{
    public double Calcular(string tipoCliente, double monto)
    {
        if (tipoCliente == "Regular")
            return monto * 0.05;

        if (tipoCliente == "VIP")
            return monto * 0.10;

        return 0;
    }
}
```

🔍 Problema

Cada vez que aparece un nuevo tipo de cliente es necesario modificar la clase, violando OCP.

☑ Código refactorizado

```
public interface IDescuento
{
    double Calcular(double monto);
}

public class DescuentoRegular : IDescuento
{
    public double Calcular(double monto) => monto * 0.05;
}

public class DescuentoVIP : IDescuento
{
    public double Calcular(double monto) => monto * 0.10;
}

public class DescuentoPremium : IDescuento
{
    public double Calcular(double monto) => monto * 0.15;
}
```

Preguntas de análisis

1. ¿Por qué el código original no es escalable?
Obliga a modificar la clase cada vez que aparece un nuevo tipo de descuento, incrementando el riesgo de introducir errores en lógica ya funcional.
2. ¿Cómo ayuda el polimorfismo?
Permite usar diferentes tipos de descuento a través de una misma interfaz, sin conocer la implementación concreta.
3. ¿Qué ventaja ofrece OCP en proyectos grandes?
Permite agregar funcionalidades (nuevos tipos de descuento) creando nuevas clases, sin tocar el código existente.

📖 Parte 3 - Principio L: Principio de sustitución de Liskov (LSP)

✗ Código "malo"

```
public class Ave
{
    public virtual void Volar()
    {
        Console.WriteLine("Volando...");
    }
}

public class Pinguino : Ave
{
    public override void Volar()
    {
        throw new Exception("Los pingüinos no vuelan");
    }
}
```

🔍 Problema

Pinguino no puede sustituir a Ave sin lanzar una excepción, rompiendo el contrato de la clase base.

☑ Código refactorizado

```
public interface IAve
{
    void Moverse();
}

public interface IAveVoladora
{
    void Volar();
}

public class Aguila : IAve, IAveVoladora
{
    public void Moverse() => Console.WriteLine("El águila se mueve");
    public void Volar() => Console.WriteLine("El águila vuela");
}

public class Pinguino : IAve
{
    public void Moverse() => Console.WriteLine("El pingüino camina y nada");
}
```

Preguntas de análisis

1. ¿Por qué el pingüino viola LSP?

Porque no puede sustituir correctamente a la clase base sin generar errores en el tiempo de ejecución.

2. ¿Qué riesgos genera esto?

Puede provocar excepciones inesperadas y comportamiento incorrecto al usar polimorfismo con la clase base.

3. ¿Cómo mejorarías la jerarquía?

Separando los comportamientos mediante interfaces específicas (IAve, IAveVoladora), de modo que cada clase implementa solo lo que realmente puede hacer.

Parte 4 — Principio I: Principio de segregación de interfaces (ISP)

✗ Código "malo"

```
public interface ITrabajador
{
    void Trabajar();
    void Comer();
}

public class Robot : ITrabajador
{
    public void Trabajar()
    {
        Console.WriteLine("Trabajando...");
    }

    public void Comer()
    {
        throw new Exception("Los robots no comen");
    }
}
```

🔍 Problema

La interfaz obliga a implementar métodos innecesarios. Robotno puede comer, pero el contrato lo fuerza a implementar el método de todas las formas, resultando en una throwcomo única salida.

☑ Código refactorizado

```
// Interfaces segregadas
public interface ITrabajador
{
    void Trabajar();
}

public interface IComedor
{
    void Comer();
}

// El humano implementa AMBAS porque las necesita
public class Humano : ITrabajador, IComedor
{
    public void Trabajar() => Console.WriteLine("Humano trabajando...");
    public void Comer()    => Console.WriteLine("Humano comiendo...");
}

// El robot SOLO implementa lo que realmente puede hacer
public class Robot : ITrabajador
{
    public void Trabajar() => Console.WriteLine("Robot trabajando...");
}
```

Preguntas de análisis

1. ¿Qué problema presenta la interfaz original?

Obliga a clases que no necesitan ciertos métodos para implementarlos de todas las formas. Robotse ve forzado a implementar Comer()con un throw, lo que es un error de diseño disfrazado de excepción en tiempo de ejecución.

2. ¿Cómo mejora el diseño del ISP?

Los ISP dividen las interfaces grandes en contratos más pequeños y específicos. Cada clase implementa solo lo que realmente necesita, eliminando implementaciones vacías o con throw.

3. ¿Qué ventajas ofrece dividir interfaces?

- **Menos acoplamiento** : los cambios en IComedorno afectan a Robot.

- **Mayor legibilidad** : el contrato de cada clase es explícito desde su definición.
- **Facilita las pruebas** : se puede simular solo la interfaz necesaria.
- **Extensibilidad** : se pueden agregar nuevas interfaces (IConductor, IDormilón) sin romper código existente.

📖 Parte 5 — Principio D: Principio de inversión de dependencia (DIP)

✗ Código "malo"

```
public class MySQLDatabase
{
    public void Guardar()
    {
        Console.WriteLine("Guardando en MySQL");
    }
}

public class UsuarioService
{
    private MySQLDatabase db = new
MySQLDatabase();
    public void CrearUsuario()
    {
        db.Guardar();
    }
}
```

🔍 Problema

UsuarioService depende directamente de MySQLDatabase. Cambiar de base de datos implica modificar el servicio, violando tanto DIP como OCP.

☑ Código refactorizado

```
// 1. Abstracción
public interface IDatabase
{
    void Guardar(string datos);
}

// 2. Implementaciones concretas
public class MySQLDatabase : IDatabase
{
    public void Guardar(string datos)
        => Console.WriteLine($"[MySQL] Guardando: {datos}");
}

public class PostgreSQLDatabase : IDatabase
{
    public void Guardar(string datos)
        => Console.WriteLine($"[PostgreSQL] Guardando: {datos}");
}

public class InMemoryDatabase : IDatabase // para pruebas unitarias
{
    public void Guardar(string datos)
        => Console.WriteLine($"[InMemory] Guardando en memoria: {datos}");
}

// 3. Módulo de alto nivel depende de la ABSTRACCIÓN
public class UsuarioService
{
    private readonly IDatabase _db;

    public UsuarioService(IDatabase db) // Inyección por constructor
    {
        _db = db;
    }

    public void CrearUsuario(string nombre)
    {
        _db.Guardar($"Usuario: {nombre}");
    }
}
```


Preguntas de análisis

1. ¿Por qué el acoplamiento es un problema?

Cuando `UsuarioService` instancia `MySQLDatabase` con `new`, quedan soldados. Cambiar de base de datos obliga a modificar el servicio, lo que puede introducir errores en la lógica que ya funcionaba.

2. ¿Qué ventajas ofrece dependiendo de abstracciones?

- El servicio no sabe ni le importa qué base de datos usa.
- Se puede sustituir la implementación sin modificar el código de negocio.
- Agregar `MongoDatabase` solo requiere implementar `IDatabase`.

3. ¿Cómo ayuda DIP en pruebas unitarias?

Permite inyectar una implementación falsa (`InMemoryDatabase`, un simulacro o stub) en lugar de una base de datos real. Las pruebas son **rápidas, aisladas y sin dependencias externas**.

Repositorio

<https://github.com/molinaenmanuel32/Solid.git>

Reflexión Final

¿Cuál principio SOLID consideras más difícil?

Considero que el principio **Liskov Substitution Principle (LSP)** es uno de los más difíciles de aplicar correctamente. Aunque la herencia parece sencilla, diseñar clases hijas que puedan sustituir completamente a sus clases padre sin alterar el comportamiento del programa requiere una buena comprensión del dominio y de las relaciones entre objetos.

¿Cuál crees que aporta más valor?

El principio que considero más valioso es el **Single Responsibility Principle (SRP)**. Mantener una única responsabilidad por clase hace que el código sea más fácil de entender, modificar y probar. Además, ayuda a prevenir muchos problemas de diseño desde las primeras etapas del desarrollo.

¿Cómo cambiaría tu manera de programar después de esta actividad?

Después de estudiar SOLID, intentaría planificar mejor la estructura de mis clases antes de comenzar a programar. También procuraría separar responsabilidades, utilizar interfaces cuando sea necesario y evitar dependencias innecesarias. Esto permitiría crear aplicaciones más organizadas y fáciles de mantener en el futuro.

¿Qué diferencias notas entre un código “rápido” y un código bien diseñado?

Un código "rápido" suele enfocarse en resolver el problema inmediato, aunque muchas veces termina siendo difícil de mantener, modificar o ampliar. Por otro lado, un código bien diseñado requiere más planificación al inicio, pero resulta más claro, reutilizable, escalable y menos propenso a errores. A largo plazo, el código bien diseñado ahorra tiempo y esfuerzo durante el mantenimiento y la evolución del proyecto.